ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ГАРЫН АВЛАГА

Үйлдлийн систем /F.CSM302/

Сэдэв: "The Producer-Consumer Problem. Programming Project 4 (7-р бүлэг), Buffer animation"

Хуваарилалтын код: II-4-12

Хичээл заасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн баг: А.Эрхэмтөр /B241940029/
/B231910039/
/B232270802/
/B242270031/
/B242270107/

Огноо: 2026 оны 10 дугаар сарын 19

ГАРЧИГ

НЭГ. *Хэрэглэгчийн заавар* 2
1.1 Программын зорилго 2
1.2 Дэмжих орчин 2
1.3 Суулгах, нээх 2
1.4 Эхлэлийн цэс 2
1.5 Хичээл ба заавар 3
1.6 Түвшингүүд 4
1.7 Чөлөөт горим ба тохиргоо 5
1.8 Ажиллуулах, зогсоох, reset 6
1.9 Дүрслэлийн тэмдэг, өнгө, төлөв 7
1.10 Жишээ 8
1.11 Түгээмэл алдаа ба шийдэл 11

ХОЁР. *Operational заавар* 11
2.1 Repository clone 11
2.2 OS, compiler/runtime, version 12
2.3 Dependency суулгах 12
2.4 Build ба Run 12
2.5 Үндсэн тестийн өгөгдөл 12
2.6 Нэмэлт boundary ба error тест 12
2.7 Хүлээгдсэн үр дүн 13
2.8 Known issues 13

НЭГ. ХЭРЭГЛЭГЧИЙН ЗААВАР

1.1 Программын зорилго

Сурах бичгийн 7-р бүлгийн Producer-Consumer (bounded buffer) төслийг C хэл дээр pthreads, POSIX semaphore (`empty`, `full`), `mutex`-ээр хэрэгжүүлсэн. Producer thread-үүд санамсаргүй тоо (0-99) тойрог буферт тавьж, consumer thread-үүд авна. Browser дээрх дэлгэц нь **бодит C программын** thread бүрийн үйлдлийг (унтах, semaphore хүлээх, тавих, авах) буферийн анимаци болон thread бүрийн timeline хэлбэрээр харуулна. Дэлгэц зөвхөн харуулдаг, бүх синхрончлол C core дотор явагдана.

1.2 Дэмжих орчин

- macOS (Apple Silicon / Intel), Linux (Ubuntu 22.04+), Windows дээр **WSL2 Ubuntu** дотор.
- Browser: Chrome, Safari, Firefox-ийн сүүлийн хувилбар.
- Windows дээр шууд (WSL-гүй) ажиллахгүй: `pc.c` нь pthreads ба POSIX semaphore ашигладаг.

1.3 Суулгах, нээх

```
git clone https://github.com/khuslenshi/os-bd1-producer-consumer.git
cd os-bd1-producer-consumer
npm install
npm run dev
```

Дараа нь browser-т **http://localhost:3000**-ийг нээхэд эхлэлийн цэс гарна. `npm run dev` нь эхлээд C core-ийг (`src/core/pc`) автоматаар build хийнэ.

1.4 Эхлэлийн цэс

Нээхэд тоглоомын цэс шиг эхлэлийн дэлгэц гарна (Зураг 1): голд гарчиг, доор нь 2 том товч нэг мөрөнд (Хичээл ба заавар, Түвшингүүд), хоёр дахь мөрөнд бүтэн өргөнтэй Тухай товч. Гарчгийн дээрх 5 нүд нь жинхэнэ C core-ийн жижиг run бөгөөд тасралтгүй эргэлдэнэ. Гараас: `←` `→` (эсвэл `↑` `↓`) сонгох, `Enter` нээх, `1`-`2` горим, `3` Тухай, бусад дэлгэцээс `Esc` дарж цэс рүү буцна.

Хүснэгт 1. Цэсний хэсгүүд

Цэс	Хаяг	Юу хийх вэ
Хичээл ба заавар	`/learn`	3 оршил, 12 алхамтай дуут хичээл (1.5)
Түвшингүүд	`/levels`	3 даалгавар, run дуусахад асуулт (1.6)
Чөлөөт горим	`/play`	Түвшингүүд хуудасны **Өөрийн туршилт** картаар нээгдэнэ; бүх тохиргоог өөрөө сонгоно (1.7)
Тухай	`/about`	Төсөл, баг, C core, Next.js route, browser хэрхэн холбогддог, гол ойлголтууд



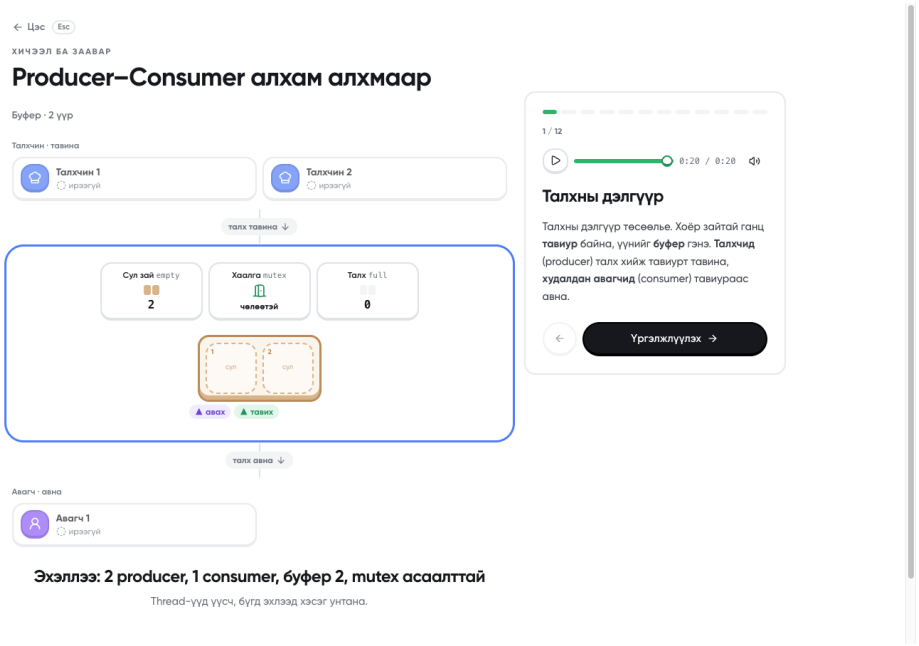
Зураг 1. Эхлэлийн цэс

1.5 Хичээл ба заавар

Эхлээд 3 оршил слайд (тавтай морил, юу сурах, хэрхэн ашиглах) дуут уншлагатай гарна. Энэ хооронд хичээлийн бодит run ар талд бичигдэж, **Хичээл эхлүүлэх** товч бэлэн болмогц идэвхжинэ.

- Эхний 5 алхам ойлголтыг заана: талхны дэлгүүрийн жишээ, яагаад буфер хэрэгтэй, гурван асуудал (дүүрэн, хоосон, race condition), компьютер дээрх буфер, `empty`/`full`/`mutex` багаж.
- Дараагийн 7 алхам бодит run-ийг алхам алхмаар тоглуулж, тухайн хэсгийг цэнхэр хүрээгээр тодруулна. Алхам хооронд бодит event-үүд нэг нэгээрээ тоглогдоно (тавих/авах бүрт 0.7 с), энэ үед **Алгасах** товчоор шууд дараагийн алхам руу очно.
- Буферийн дүрслэл алхмаас хамаарч өөрөө солигдоно: эхлээд Талх, дараа нь Хоолой, Массив.
- 5 алхамд асуулт байгаа, зөв хариулсны дараа үргэлжилнэ.
- Алхам бүрийг дуу хоолойгоор уншина (урьдчилан бичсэн `public/audio/learn/*.m4a`): тоглуулах/зогсоох товч, slider-аар гүйлгэж болно. Тавих/авах/хүлээх үед болон хариултад богино дуу гарна.

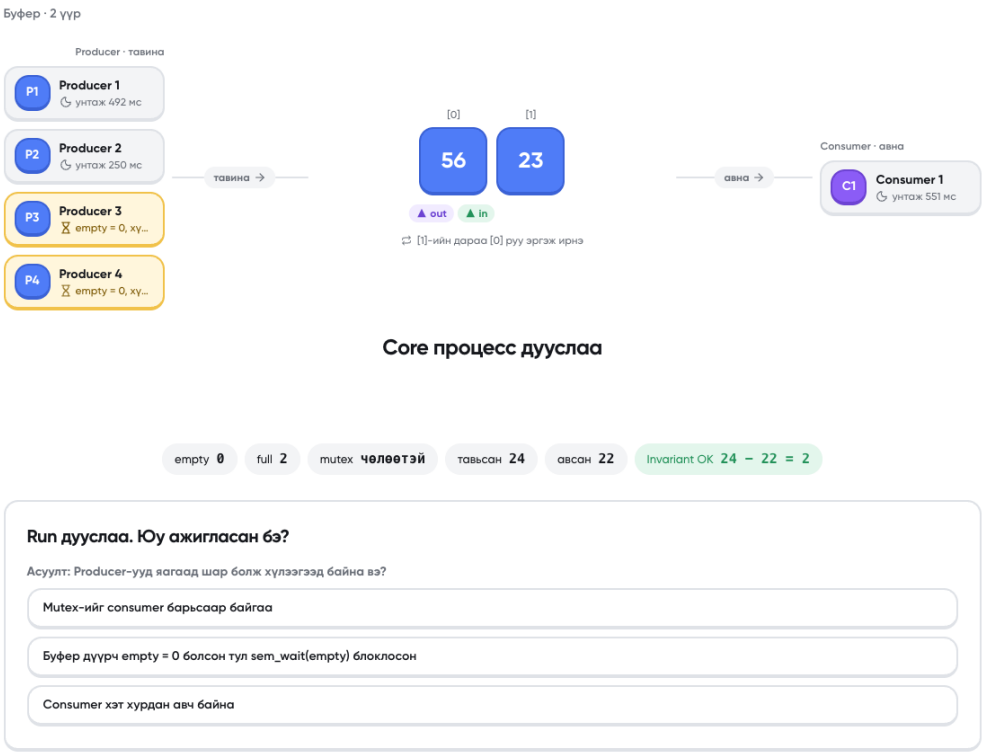
- `/learn`-ийг шууд хаягаар нээвэл browser эхний уншлагыг хааж магадгүй, тэр үед тоглуулах товчийг дарна.



Зураг 2. Хичээл ба заавар: талхны дэлгүүрийн жишээ

1.6 Түвшингүүд

3 даалгавар: Буфер дүүрэх, Буфер хоосрох, Race condition. Тохиргоо нь түгжигдсэн (зөвхөн дүрслэлийг сольж болно). Start дараад ажигласны дараа run дуусахад асуулт гарна (Зураг 3). Зөв хариулсан түвшин "Дууссан" гэж тэмдэглэгдэнэ (тухайн browser-т хадгалагдана). Тоглуулах үед тавих/авах/хүлээх бүрт богино дуу, race үед "буруу" дуу, асуултын хариултад зөв/буруугийн дуу гарна.



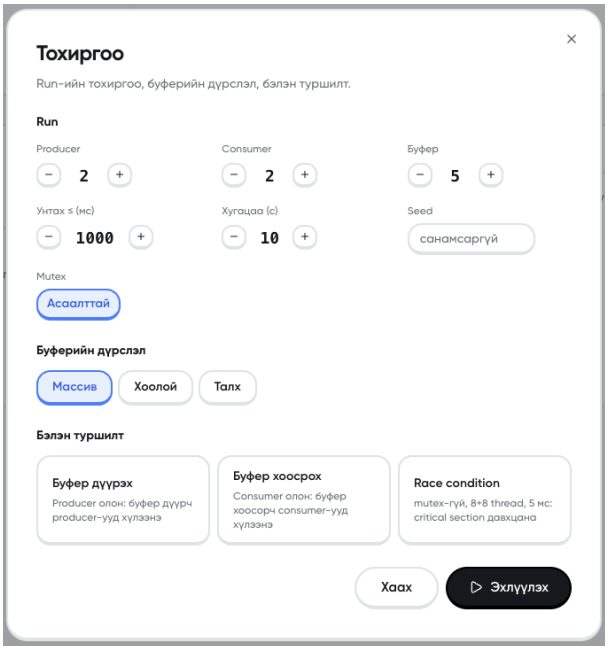
Зураг 3. Түвшин 1 дууссаны дараах асуулт

1.7 Чөлөөт горим ба тохиргоо

Бүх тохиргоо **Тохиргоо** товчоор нээгдэх цонхонд байна (Зураг 4): run-ий тоонууд, seed, mutex, буферийн дүрслэл (Массив / Хоолой / Талх), бэлэн туршилтууд. Товчны хажууд одоогийн тохиргооны хураангуй (жишээ нь `P2 · C2 · 5 үүр · mutex · Массив`) харагдана. Тоо бүрийг - / + товчоор эсвэл шууд бичиж өөрчилнө. Хүрээнээс гадуур утга оруулбал тоо улаан болж, Start дарахад алдааны тайлбар гарна (Зураг 10).

Хүснэгт 2. Ролт ба тохиргоо

Талбар	Утга	Хүрээ	Анхдагч
Producer	producer thread-ийн тоо	0-16	2
Consumer	consumer thread-ийн тоо	0-16	2
Буфер	тойрог буферийн үүрийн тоо	1-16	5
Унтах ≤ (мс)	thread бүр үйлдэл хооронд 0-оос энэ хүртэл санамсаргүй унтана	0-5000	1000
Хугацаа (с)	run хэдэн секунд үргэлжлэх	1-60	10
Seed	санамсаргүй тооны эх утга; ижил seed = ижил унтах хугацаа	0-2147483647, хоосон бол санамсаргүй	хоосон
Mutex	унтраавал critical section хамгаалалтгүй болж race condition гарна	асаах/унтраах	асаалттай



Зураг 4. Тохиргоо цонх

1.8 Ажиллуулах, зогсоох, reset

- **Start** (эсвэл Тохиргоо цонхны **Эхлүүлэх**): тохиргоогоор шинэ run эхлүүлнэ. Цонхны бэлэн туршилтын товч (Буфер дүүрэх / Буфер хоосрох / Race condition) тохиргоог бөглөөд шууд эхлүүлж, цонхыг хаана.
- **Stop**: ажиллаж буй C процессыг зогсооно, тэр хүртэлх event-үүд үлдэнэ.
- **Reset**: run-ийг зогсоож дэлгэцийг цэвэрлэнэ.

- Timeline дээр **дарж эсвэл чирж** аль ч агшин руу очно. ← → нэг event, Shift+←/→ 10 event, Space pause/play.
- **Доод bar** (дэлгэцийн доор үргэлж харагдана): видео тоглуулагч шиг slider (зүүн талд одоогийн хугацаа, баруун талд нийт хугацаа), **0.1× 0.25× 0.5× 1×** тоглуулах хурд (анхдагч 0.25×, 1× = бодит хугацаа), **Replay**, **Өмнөх**, **Тоглуулах / Түр зогсоох**, том хар **Дараагийн алхам** товч (нэг event урагшилна).
- **Event log** (өргөн дэлгэц дээр баруун талд, нарийн дэлгэц дээр timeline-ий доор): тоглогдсон event бүр доороос нэмэгдэнэ. Мөр дээр дарж тэр агшин руу очно. **JSONL** товчоор бүх event-ийг файлаар татна.
- C core бодит хугацаагаар ажилладаг, дэлгэц түүнийг сонгосон хурдаар араас нь тоглуулна. Core илгээсэн боловч хараахан тоглогдоогүй хэсэг timeline дээр бүдэг харагдана. Тавих/авах бүр дэлгэц дээр дор хаяж 0.6 с, хүлээх 0.4 с харагдана (0.25× үед, бусад хурдад харьцаагаар). 600-аас олон event-тэй run (жишээ нь Түвшин 3) зөвхөн хугацаагаар тоглогдож, хурд 0.1×-д түгжигдэнэ.
- Timeline-ий баруун дээд буланд **2 с / 5 с / Бүгд** цонх: анхдагч нь 2 с; 2 с, 5 с үед timeline playhead-ийг дагаж гүйнэ.

1.9 Дүрслэлийн тэмдэг, өнгө, төлөв

Буферийн дүрслэл. Тохиргоо цонхны "Буферийн дүрслэл" хэсгээс **Массив / Хоолой / Талх** гурвын аль нэгийг сонгоно (Хичээл дээр алхмаас хамаарч өөрөө солигдоно). Гурвуулаа ижил өгөгдлийг харуулна.

- **Массив** (номын Figure 7.1 шиг): буфер = нүдний мөр (circular array). Цэнхэр нүдэнд item байна, саарал нүд хоосон, яг одоо хандсан нүд ногоон болж дээш өргөгдөнө (race бол улаан). **▲in** (ногоон) нь producer-ийн дараагийн тавих нүд, **▲out** (нил ягаан) нь consumer-ийн дараагийн авах нүд. Сүүлийн нүдний дараа [0] руу эргэж ирнэ.
- **Хоолой** (FIFO): item зүүн талаас орж, баруун талаас гарна. Хамгийн баруун талынх нь хамгийн эрт орсон тул эхэлж авагдана. Тасархай тойрог = сул зай (`empty`), цэнхэр = item (`full`).
- **Талх**: талхчин = producer, авагч = consumer, тавиур = буфер (модон хайрцагт нүд бүр нэг үүр, хоосон нүд тасархай хүрээтэй "сул", талх дээр item-ийн дугаар). Дээрх самбарууд: **Сул зай** = `empty`, **Хаалга** = `mutex` (онгорхой = чөлөөтэй, хаалттай бол дотор нь хэн байгаа), **Талх** = `full`.

Thread карт ба сум.

- Thread бүр нэг карт: өнгөт avatar (producer цэнхэр, consumer ягаан), нэр, одоогийн төлөв icon-той (сар = унтаж, элсэн цаг = хүлээж). Саарал = унтаж байна, шар = semaphore хүлээж байна, цайвар цэнхэр/ягаан = тавьж/авч байна (гартаа item-ийн дугаартай), хар хүрээ + цоож = mutex эзэмшиж байна. 6-аас олон thread үед карт жижиг chip болно.
- Item тавихад producer-оос нүд рүү бөмбөлөг нисэж очно, авахад нүднээс consumer руу нисэнэ.

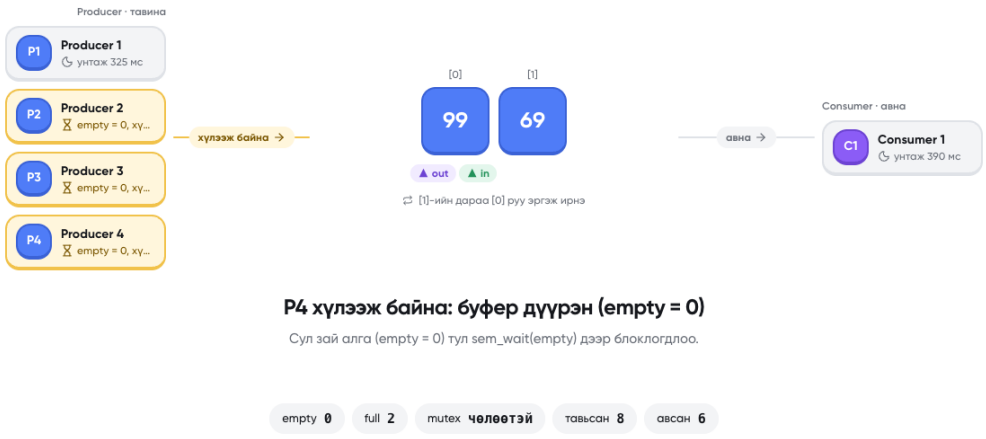
- Producer, буфер, consumer гурвын хооронд **тавина** ба **авна** сум байна. Тавих/авах агшинд тухайн сум цэнхэр/ягаан болж item-ийн дугаарыг харуулна, тал нь хүлээж байвал шар "хүлээж байна" болно.
- Буферийн доор том үсгээр одоогийн event, түүний доор "яагаад" гэсэн нэг мөр тайлбар (аль semaphore блоклосон, mutex-ийг хэн барьж байгаа) гарна. Доор нь `empty`, `full`, `mutex`, тавьсан, авсан тоо болон төгсгөлд invariant.

Хүснэгт 3. Thread timeline-ий өнгө

Өнгө	Утга
саарал зурвас	унтаж байна (`sleep`)
шар зурвас	semaphore хүлээж байна (`wait_empty`: буфер дүүрэн, `wait_full`: буфер хоосон)
цэнхэр ↓27	produce: 27-г буферт тавив
нил ягаан ↑27	consume: 27-г буферээс авав
хар P2 (mutex мөр)	mutex-ийг тэр агшинд эзэмшсэн thread
улаан	race: тухайн агшинд өөр thread зэрэг critical section-д байсан
`count` мөр	буферт байгаа item-ийн тоо цаг хугацааны дагуу

1.10 Жишээ

Жишээ 1. Буфер дүүрэх (P=4, C=1, буфер=2, ≤600 мс, 8 с, seed 7). Producer олон, consumer ганц тул буфер хурдан дүүрнэ: producer-ууд шар (`empty = 0, хүлээж`) болно. Timeline дээр P1-P4-ийн мөрөнд урт шар зурвас харагдана. Төгсгөлд `Invariant OK`.



Зураг 5. Жишээ 1. Буфер дүүрэх

Жишээ 2. Буфер хоосрох (P=1, C=4, буфер=5, "Хоолой" дүрслэлээр). Consumer олон тул буфер ихэнхдээ хоосон: хоолой бүхэлдээ тасархай тойрог (`full = 0`), consumer-ууд шар (`full = 0, хүлээж`).

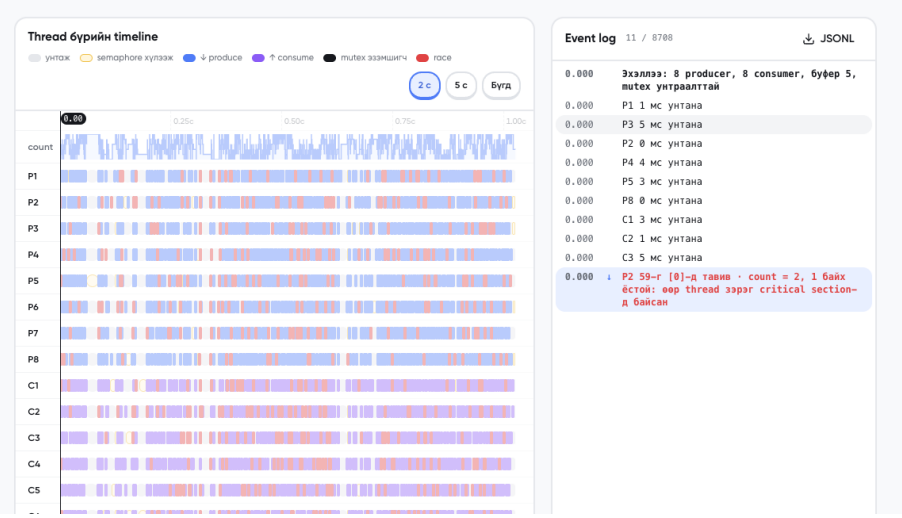


Зураг 6. Жишээ 2. Буфер хоосрох

Жишээ 3. Race condition (P=8, C=8, буфер=5, ≤5 мс, 1 с, mutex унтраалттай). Critical section хамгаалалтгүй тул timeline дээр олон улаан тэмдэг гарна. 600-аас олон event-тэй тул дээр нь шар анхааруулга гарч, хурд 0.1×-д түгжигдэнэ. Race хэсгийн **Эхнийх**, **Дараагийнх** товчоор race-уудыг нэг нэгээр нь үзэхэд доор нь тайлбар гарна, жишээ нь "count = 2, 1 байх ёстой". Зарим run-д төгсгөлд `Invariant БУРУУ` гарна, энэ нь шинэчлэл алдагдсаныг (lost update) баталгаажуулна. Mutex-тэй ижил тохиргоонд race 0 гарна.

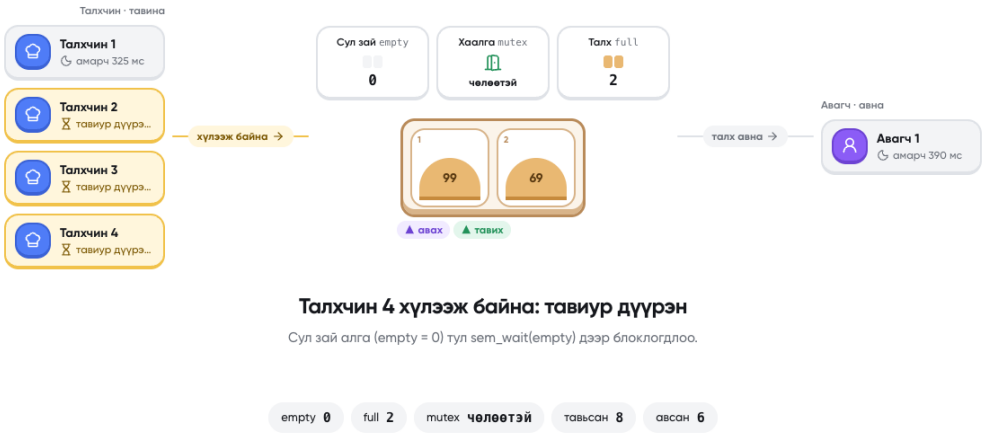


Зураг 7. Жишээ 3. Race condition, эхний race дээр



Зураг 8. Race-ийн timeline ба Event log

Жишээ 4. Талхны дэлгүүрээр ("Талх" дүрслэл, Буфер дүүрэх тохиргоо). Талхчид шар картаар "тавиур дүүрэн, хүлээж", Сул зай 0, Талх 2.

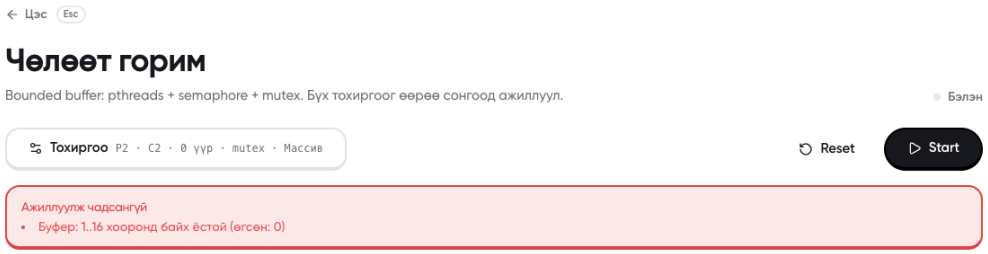


Зураг 9. Жишээ 4. Талх дүрслэл

1.11 Түгээмэл алдаа ба шийдэл

Хүснэгт 4. Түгээмэл алдаа

Алдаа	Шалтгаан	Шийдэл
`Буфер: 1..16 хооронд байх ёстой`	хүрээнээс гадуур утга	талбарын хүрээнд утга оруулна (Зураг 10)
`C core build хийгдээгүй байна`	`src/core/pc` байхгүй	`make -C src/core` эсвэл `npm run dev`-ээр дахин эхлүүлнэ
`Зэрэг 4-өөс олон run...`	4-өөс олон tab зэрэг run хийж байна	бусад tab-ийн run-ийг Stop хийнэ
`npm: command not found`	Node суугаагүй	Node.js 20+ суулгана
`make: cc: No such file`	C compiler байхгүй	macOS: `xcode-select --install`, Ubuntu/WSL: `sudo apt install build-essential`
Port 3000 эзэлсэн	өөр программ ашиглаж байна	`npm run dev -- -p 3100`



Зураг 10. Буруу оролт: буфер 0 үед алдааны мэдээлэл

ХОЁР. OPERATIONAL ЗААВАР

2.1 Repository clone

```
git clone https://github.com/khuslenshi/os-bd1-producer-consumer.git
cd os-bd1-producer-consumer
```

2.2 OS, compiler/runtime, version

Хүснэгт 5. Туршсан орчин

Орчин	OS	Compiler	Node	Python
Хөгжүүлэгчийн Mac	macOS 27.0.1, arm64	Apple clang 21.0.0	26.5.1 (npm 11.17.0)	3.14.6
GitHub Actions	ubuntu-latest	gcc (`-Werror`)	24	3
GitHub Actions	macos-latest	Apple clang (`-Werror`)	24	3

Шаардлага: C compiler + `make`, Node.js 20 эсвэл түүнээс дээш. Python 3 нь зөвхөн `tests/run_tests.py`-д хэрэгтэй.

2.3 Dependency суулгах

```
npm install          # for development
npm ci               # CI, exact lockfile versions
```

Бүх package-ийн хувилбар `package.json`-д яг тогтоогдсон, `package-lock.json` repo-д бий.

2.4 Build ба Run

```
npm run dev                      # development, http://localhost:3000
npm run build && npm start       # production
make -C src/core                 # C core only
# core alone, prints one JSON line per event:
./src/core/pc 10 3 2 --buffer 5 --sleep-max 1000
```

2.5 Үндсэн тестийн өгөгдөл

```
npm test
```

- `src/lib/model.test.ts` (3 тест): event-ийг timeline болгох, race илрүүлэлт. Үүний нэг нь **бодит core**-ийг ажиллуулна: mutex-тэй (8+8 thread, 5 мс) run-д race 0, mutex-гүй run-д race > 0 байх ёстой.
- `tests/run_tests.py` (18 тест): T1-T4 үндсэн (1P+1C, буфер дүүрэх, хоосрох, invariant).

2.6 Нэмэлт boundary ба error тест

- `tests/run_tests.py`: B1-B6 (буфер=1, sleep=0, 100+100 thread, буфер=64, 0 consumer, 0 producer), E1-E7 (буруу аргумент), S1 (SIGTERM-ээр 1 с-д зогсох).
- `/api/run`-ийн шалгалт (CI-д):

```
curl -s 'http://localhost:3000/api/run?buffer=0'
#   -> HTTP 400 + error message
curl -sN 'http://localhost:3000/api/run?duration=1&producers=2&consumers=2&
    buffer=5&sleepMax=50&mutex=1' | tail -2
#   -> ... "invariant_ok":true
```

2.7 Хүлээгдсэн үр дүн

- `npm test`: `pass 3`, дараа нь `18/18 PASS`.
- Mutex-тэй бүх run: `Invariant OK`, race 0.
- Mutex-гүй race туршилт: race тэмдэг > 0, зарим run-д `Invariant БУРУУ`.
- GitHub Actions: `Build & test (ubuntu-latest)`, `Build & test (macos-latest)` ногоон.

2.8 Known issues

- Нэг run-д 20000-аас олон event ирвэл core-ийг зогсоож анхааруулна (унтах 0-5 мс, олон thread үед).
- Event олон (600+) үед timeline-ий шошго нуугдаж нарийн зураас болно, item нисэх анимаци унтарна. Ойрхон event-үүдийн шошгыг харахын тулд 2 с цонх ашиглана.
- Mutex-гүй үед log-ийн дараалал бодит дараалалтай яг таарахгүй (log бичих нь critical section-оос гадна), тиймээс race-ийг `count`-ийн зөрүүгээр илрүүлдэг.
- Нэг машин дээр зэрэг 4 хүртэл run.